

# TRACE: Trust, Runtime Attestation, and Compliance Evidence

A Portable Attestation Format for AI Agent Runtime Governance

Rishabh Poddar   Aaron Fulkerson   Imran Siddique  
OPAQUE Systems

<https://github.com/agentrust-io/trace-spec>

October 1, 2026

Technical report, version 3

## Abstract

TRACE (Trust, Runtime Attestation, and Compliance Evidence) proposes a portable record for claims about AI agent runtimes: workload identity, model, policy, data class, tool transcript, build provenance, and appraisal. It composes RATS/EAT, signatures, and transparency evidence into a format that a relying party can evaluate against its own trust policy. A valid signature establishes integrity and possession of the signing key; hardware origin additionally requires verified attestation and a binding from that evidence to the key. This technical report describes the original design and preserves the recorded software evaluation of agentrust-trace (trace-spec revision 523f8cc, whose signing code is release 0.3.0) and trace-tests 0.2.0. The evaluation measures canonicalization, Ed25519 signing and verification, record size, and fixture outcomes. It does not measure silicon certificate-chain appraisal or establish that the recorded runtime claims are true. The current normative specification and its implementation limits are maintained separately.

**Scope of this edition.** This is an author-prepared technical report, not peer reviewed and not a normative specification. It revises a manuscript originally dated June 23, 2026; the recorded experiments include later runs and have not been rerun for this edition. Architectural requirements describe intended checks, not proof that the historical SDK implements every check. Consult the linked project specification and limitations before implementing the design. Version 3 (October 1, 2026) corrects the identification of the evaluated SDK, states that the evaluated vectors and checker use the v0.1 profile URI, narrows the conformance-module descriptions to what the evaluated checks do, fixes the install command, corrects five bibliography entries, replaces a vendor model name in the worked example, and restores bold and italic type that the Tectonic build had dropped; the recorded numbers are unchanged. Version 2 corrected one sentence in the related-work discussion of transparency logs, which implied that log registration makes omitted events detectable.

## Contents

|          |                                 |          |
|----------|---------------------------------|----------|
| <b>1</b> | <b>Introduction</b>             | <b>3</b> |
| <b>2</b> | <b>Background</b>               | <b>3</b> |
| 2.1      | Remote Attestation (RATS)       | 3        |
| 2.2      | Entity Attestation Tokens (EAT) | 3        |
| 2.3      | TEE Attestation                 | 4        |
| 2.4      | Agent Identity                  | 4        |
| 2.5      | The gap                         | 4        |
| <b>3</b> | <b>The Trust Record</b>         | <b>4</b> |
| 3.1      | Logical schema                  | 4        |
| 3.2      | Claim structure                 | 4        |
| 3.3      | Wire format                     | 5        |

|           |                                              |           |
|-----------|----------------------------------------------|-----------|
| 3.4       | Signature binding . . . . .                  | 5         |
| 3.5       | Freshness . . . . .                          | 5         |
| 3.6       | Worked example . . . . .                     | 5         |
| 3.7       | External execution evidence . . . . .        | 6         |
| <b>4</b>  | <b>Roles and Protocol</b>                    | <b>6</b>  |
| 4.1       | Roles . . . . .                              | 6         |
| 4.2       | Key hierarchy . . . . .                      | 7         |
| 4.3       | Single-organization flow (Phase 1) . . . . . | 7         |
| 4.4       | Cross-organization flow (Phase 2) . . . . .  | 7         |
| <b>5</b>  | <b>Verification Procedure</b>                | <b>7</b>  |
| 5.1       | Result object . . . . .                      | 8         |
| <b>6</b>  | <b>Profiles</b>                              | <b>8</b>  |
| 6.1       | The cMCP profile . . . . .                   | 8         |
| 6.2       | Extensibility . . . . .                      | 9         |
| <b>7</b>  | <b>Conformance and Reproducibility</b>       | <b>9</b>  |
| 7.1       | Conformance suite . . . . .                  | 10        |
| 7.2       | Transparency registry . . . . .              | 10        |
| <b>8</b>  | <b>Evaluation</b>                            | <b>10</b> |
| 8.1       | Canonicalization determinism . . . . .       | 11        |
| 8.2       | Sign and verify cost . . . . .               | 11        |
| 8.3       | Record size . . . . .                        | 12        |
| 8.4       | Conformance suite behaviour . . . . .        | 12        |
| <b>9</b>  | <b>Related Work</b>                          | <b>13</b> |
| <b>10</b> | <b>Conclusion</b>                            | <b>13</b> |

# 1 Introduction

AI agents now invoke tools, call external services, and exchange data across organizational boundaries. When a relying party (a downstream service, an auditor, or a partner organization) consumes the result of an agent workflow, it has no portable, vendor-neutral way to answer basic governance questions: which policy governed the agent, which identity was authenticated, what hardware enforced the policy, and whether the recorded evidence has been altered. Ad hoc logs are operator-controlled and therefore not trustworthy across a trust boundary.

TRACE (Trust, Runtime Attestation, and Compliance Evidence) is a signed, hardware- attestable evidence record for AI agent runtimes. It instantiates the RATS roles [Birkholz et al., 2023] (Attester, Verifier, Relying Party) and profiles the Entity Attestation Token [Lundblade et al., 2025] with the fields an agent runtime needs: the model and runtime measurements, the policy and data class in effect, the invoked-tool transcript, the authenticated agent identity, a confirmation key rooted in the TEE attestation, and an inclusion proof on an append-only transparency log. The result, a *Trust Record*, is appraisable offline, without trusting the operator that produced it.

## Contributions

- **A portable evidence record.** The Trust Record: an EAT profile that binds model, runtime, policy, data class, tool transcript, identity, a confirmation key, and a transparency receipt into one signed artifact (Section 3).
- **A role and protocol model.** How Attester, Verifier, and Relying Party interact in the RATS passport model, for single- and cross-organizational deployments (Section 4).
- **An offline verification procedure.** Signature binding, freshness, silicon-root chaining, reference-value matching, policy, transparency, and provenance, with no callback to the issuer (Section 5).
- **A reference profile and conformance suite.** cMCP [Poddar et al., 2026] produces Trust Records at the MCP boundary (Section 6), and a conformance suite checks records against the normative requirements (Section 7).

## Paper structure

Section 2 reviews RATS, EAT, TEE attestation, and agent identity. Section 3 defines the Trust Record. Section 4 gives the role model and protocol. Section 5 specifies verification. Section 6 describes profiles, with cMCP as the reference instance. Section 7 describes the conformance suite, Section 9 surveys related work, and Section 10 concludes.

# 2 Background

## 2.1 Remote Attestation (RATS)

The RATS architecture [Birkholz et al., 2023] separates three roles: the *Attester* produces evidence about a target environment; the *Verifier* appraises that evidence against policy and reference values to produce an attestation result; and the *Relying Party* consumes the result to make a trust decision. TRACE maps the agent runtime to the Attester; the Trust Record carries both the runtime’s Evidence and the Verifier’s appraisal (the appraisal claim, in the EAR format) so a Relying Party can act on it offline. RATS describes two topologies: the *background-check* model, in which the Relying Party forwards Evidence to a Verifier at decision time, and the *passport* model, in which the Attester obtains an appraised result up front and presents it directly. TRACE uses the passport model: the Trust Record is self-contained and verifiable offline, so no party need call back to the issuer.

## 2.2 Entity Attestation Tokens (EAT)

EAT [Lundblade et al., 2025] standardizes a token carrying claims about an entity, including measurements, a nonce, and a confirmation (cnf) key, in a JWT or CBOR-COSE envelope. TRACE is an EAT profile under the URI tag:agenttrust-io.com,2026:trace-v0.2. It reuses the standard EAT claims (iat, nonce, cnf, the profile claim) and adds the agent-governance claims an agent runtime needs (model, policy, data\_class, tool\_transcript, build\_provenance, appraisal, transparency); these are enumerated in Section 3.

## 2.3 TEE Attestation

Confidential-computing platforms (AMD SEV-SNP [Advanced Micro Devices, Inc., 2020] and Intel TDX [Intel Corporation, 2023]) produce hardware-signed reports binding a launch measurement and a caller-supplied `report_data` field. TRACE records the measurement in runtime and binds the record’s signing key through the `cnf` claim; a profile additionally commits the key into `report_data` so the hardware report and the signing key are cryptographically linked (the cMCP profile does this with an RFC 7638 JWK thumbprint, Section 6). The `runtime.measurement` is the platform’s launch measurement (for SEV-SNP the 48-byte report measurement, for TDX the MRTD and runtime measurement registers), which a verifier checks against vendor-published Reference Integrity Manifests (RIMs) named by `runtime.rim_uri`.

## 2.4 Agent Identity

SPIFFE/SVID [SPIFFE Project, 2018] provides workload identity that TRACE carries in the `subject` claim (a SPIFFE ID or DID URI), so a record names *which* agent acted, not only *which* enclave ran. The SVID is bound to the TEE measurement, rooting identity in hardware; the companion Agent Manifest specification [Siddique, 2026] supplies the issuer keys and the manifest-to-session binding behind that identity.

## 2.5 The gap

Each layer above exists, but none, alone, lets a relying party verify agent runtime governance across a trust boundary. Static documentation (model cards, AIBOMs) diverges from deployed reality; operational logs are self-reported and mutable; hardware attestation proves the environment is genuine but says nothing about policy, data class, or tools; content provenance is silent on execution. No open, portable, vendor-neutral artifact binds *what ran, under what policy, on what data, invoking which tools* into one record a third party can appraise offline. TRACE closes that gap.

# 3 The Trust Record

The unit of evidence in TRACE is the *Trust Record*: a single signed artifact that binds *what executed* (model, code, runtime), *under what policy, on what data class, invoking which tools*, optionally accompanied by attestation evidence. A record travels with the workload across hosts, clouds, and providers. Offline checks require the relevant trusted keys and evidence; a signature alone does not establish silicon provenance.

## 3.1 Logical schema

A Trust Record is an EAT [Lundblade et al., 2025] carrying the claims in Table 1. All fields are required unless marked optional. The presence of a field does not verify its contents; appraisal requires independent evidence and trust policy. Sub-records (for example per-tool-call transcripts) compose under one root envelope.

## 3.2 Claim structure

The composite claims carry the following members (optional members italicized):

- `model`: `provider`, `model_id`, *`version`*, *`weights_digest`*, *`aibom_uri`*.
- `runtime`: `platform` (one of `intel-tdx`, `amd-sev-snp`, `nvidia-h100`, `nvidia-blackwell`, `aws-nitro`, `arm-cca`, `google-confidential-space`, `tpm2`, `software-only`), `measurement` (a `sha256:/sha384:digest`), *`rim_uri`*, *`nonce`*, *`firmware_version`*.
- `policy`: `bundle_hash`, `enforcement_mode` (`enforce` | `advisory` | `silent`), *`version`*, *`policy_uri`*. `enforcement_mode` MUST default to `enforce`; `silent` MUST be configured explicitly.
- `tool_transcript`: `hash` (binds the full transcript), *`call_count`*, *`transcript_uri`*.
- `build_provenance`: `slsa_level` (0 to 3), *`builder`*, *`digest`*, *`provenance_uri`*.
- `appraisal`: `status` (`affirming` | `warning` | `contraindicated` | `none`), *`verifier`*, *`policy_ref`*, *`timestamp`*.

Table 1: Trust Record claims (TRACE v0.2). “Source primitive” names the existing standard each claim is drawn from; TRACE adds the binding into a hardware-attested envelope rather than redefining the primitive.

| Field                         | Description                                                                                        | Source primitive                      |
|-------------------------------|----------------------------------------------------------------------------------------------------|---------------------------------------|
| <code>eat_profile</code>      | Profile URI identifying a TRACE v0.2 record:<br><code>tag:agenttrust-io.com,2026:trace-v0.2</code> | EAT profile claim                     |
| <code>iat</code>              | Issued-at timestamp (Unix epoch)                                                                   | EAT standard claim                    |
| <code>subject</code>          | Workload identity (agent, tool, or model invocation)                                               | SPIFFE SVID or DID URI                |
| <code>model</code>            | Model identity, weights digest, version                                                            | EAT claim + AIBOM ref                 |
| <code>runtime</code>          | TEE measurement chain (firmware → kernel → image → workload)                                       | RATS Evidence + vendor RIM            |
| <code>policy</code>           | Bound policy-set hash and enforcement mode                                                         | Policy hash sealed to TEE measurement |
| <code>data_class</code>       | Classification of inputs and outputs                                                               | Per-call classification label         |
| <code>tool_transcript</code>  | ( <i>optional</i> ) Tool calls invoked, parameters classified, responses filtered                  | MCP / A2A transcript, bound by hash   |
| <code>build_provenance</code> | How the running code and model were built                                                          | SLSA Provenance v1.0                  |
| <code>appraisal</code>        | Verifier’s appraisal of the evidence                                                               | EAR (AR4SI)                           |
| <code>transparency</code>     | Inclusion proof on an append-only log                                                              | SCITT receipt URI                     |
| <code>cnf</code>              | Confirmation key binding the record to a TEE-held signing key                                      | EAT <code>cnf</code> claim (RFC 7800) |
| <code>signature</code>        | ( <i>optional field</i> ) Embedded signature; see Section 3.4                                      | JWS / COSE over canonical JSON        |

- `cnf`: a single jwk (kty, and the key material required by the key type, e.g. `crv/x` for OKP, `crv/x/y` for EC).

The three questions a deployer must answer map onto these claims: `runtime+model+build_provenance` answer *what ran*; `tool_transcript` answers *what it did*; `policy+data_class` answer *what rules were in force*.

### 3.3 Wire format

The envelope is an EAT [Lundblade et al., 2025]: a JWT in JSON-friendly contexts, or a CWT/CBOR-COSE in constrained or high-throughput contexts. The profile URI is `tag:agenttrust-io.com,2026:trace-v0.2`.

### 3.4 Signature binding

Every Trust Record MUST be cryptographically bound by a signature over its canonical form, made by the key in `cnf`. Canonicalization is RFC 8785 (JCS): ad-hoc sorting such as `json.dumps(sort_keys=True)` is insufficient because it diverges from JCS on non-ASCII strings and IEEE 754 number serialization. The binding may be *embedded* (the top-level `signature` field, `base64url`, over the record with `signature` absent) or *enveloping* (a JWS, a COSE\_Sign1, or cMCP’s RuntimeClaim). Each profile declares which it uses. A record with no verifiable binding is not a Trust Record; verifiers MUST reject it, and schema validity alone confers no trust. JWT contexts use ES256, ES384, or EdDSA; COSE contexts use COSE\_Sign1 or COSE\_Sign.

### 3.5 Freshness

Records MUST carry `iat`, and verifiers MUST enforce a maximum age (default 24 hours; a profile MAY override). For online verification, a verifier MAY supply a challenge nonce that the issuer echoes in `runtime.nonce`; when a challenge was issued, a record that omits or mismatches it MUST be rejected.

### 3.6 Worked example

A Trust Record in the readable (JWT) form, abbreviated:

```

1 {
2   "eat_profile": "tag:agentrust-io.com,2026:trace-v0.2",
3   "iat": 1750676142,
4   "subject": "spiffe://trust.example.org/agent/payments-processor/prod",
5   "model": { "provider": "example-provider", "model_id": "example-model",
6             "version": "20251001", "weights_digest": "sha256:a3f8d2c1..." },
7   "runtime": { "platform": "amd-sev-snp", "measurement": "sha384:c9e4b1d2...",
8               "rim_uri": "https://kdsintf.amd.com/vcek/v1/Milan/..." },
9   "policy": { "bundle_hash": "sha256:b2c3d4e5...", "enforcement_mode": "enforce",
10              "version": "1.2.0" },
11  "data_class": "confidential",
12  "tool_transcript": { "hash": "sha256:d4e5f6a7...", "call_count": 3 },
13  "build_provenance": { "slsa_level": 2,
14                       "builder": "https://github.com/slsa-framework/slsa-github-generator",
15                       "digest": "sha256:e5f6a7b8..." },
16  "appraisal": { "status": "affirming", "verifier": "https://trust-authority.example.org" },
17  "transparency": "https://registry.agentrust-io.com/claim/trace-2026-06-23T09:15:42Z-f2a8d1",
18  "cnf": { "jwk": { "kty": "EC", "crv": "P-256", "x": "MEkwEw...", "y": "GHkVPy..." } }
19 }

```

The signature is carried per the binding the profile declares (Section 3.4); the cMCP profile wraps this record in a signed RuntimeClaim (Section 6).

### 3.7 External execution evidence

A deployment may attach independent, out-of-band receipts to individual audit-chain entries (for example, a signed assertion from a safety controller confirming or rejecting an actuation). The Trust Record commits the audit chain by hash; the receipts live inside that chain, not in the record. A receipt carries an issuer identity (*issuer*), a key reference (*issuer\_key\_id*), a signature over its canonical fields (*signature*), a content digest (*evidence\_hash*), a type tag (*evidence\_type*), and a binding to the tool call (*linked\_call\_id*). A verifier holding the issuer's key verifies the signature over the canonical (JCS) receipt fields and asserts that *linked\_call\_id* matches the enclosing entry; a failure invalidates the record. A receipt whose issuer key is unknown is *unverified, not invalid*: the gateway-produced evidence stands, and the verifier surfaces an advisory status. TRACE binds the receipt into the audit chain; it does not certify that the physical action occurred or met any functional-safety standard: that claim belongs to the issuer.

## 4 Roles and Protocol

### 4.1 Roles

TRACE instantiates the three RATS roles [Birkholz et al., 2023]. The *Attester* is the agent runtime: it holds the TEE-bound signing key, gathers Evidence (the platform measurement chain), and issues a Trust Record. The *Verifier* appraises that Evidence against reference values and policy and emits an appraisal (the record's appraisal claim, in the EAR format). The *Relying Party* (a downstream service, an auditor, or a partner) consumes the record and the appraisal to make a trust decision.

TRACE follows the RATS *passport* model: the Attester obtains its evidence and ships a self-contained Trust Record that travels with the workload. Any party verifies it offline, with no callback to the issuer and no vendor in the trust path beyond the silicon root and the transparency-log operator (Section 5).

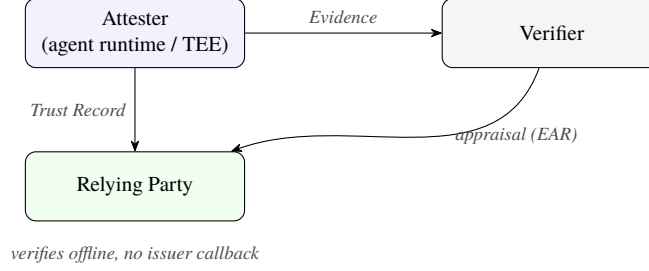


Figure 1: TRACE in the RATS passport model. The Attester ships a self-contained Trust Record; the Verifier’s appraisal travels in the record so the Relying Party appraises it without contacting the issuer.

## 4.2 Key hierarchy

Signing keys form a chain rooted in silicon: silicon root key (vendor-managed, hardware-bound) → platform attestation key (Intel TDX Quote-signing key, AMD VCEK/VLEK, NVIDIA NRAS) → workload attestation key (TEE-bound, ephemeral) → record-signing key (per workload, optionally per session). The record-signing key is the one named in `cnf`; workload-level keys SHOULD rotate at TEE-image boundaries, and verifiers MUST consult current revocation status at verification time.

## 4.3 Single-organization flow (Phase 1)

1. **Startup attestation.** The runtime launches inside a TEE, derives its record-signing key, and obtains a platform attestation report binding that key.
2. **Issuance.** For each governed execution (an invocation, or a session), the runtime assembles a Trust Record (`model`, `runtime`, `policy`, `data_class`, `tool_transcript`, `build_provenance`), signs it (Section 3.4), and optionally registers it on the transparency log, filling transparency.
3. **Appraisal and consumption.** A Verifier appraises the record and a Relying Party acts on it, both offline.

Issuance granularity is profile-defined: the cMCP profile issues one record per session, at session close, binding that session’s audit chain; a profile that needs finer evidence may issue per invocation. In all cases the subject is the workload’s SPIFFE identity [SPIFFE Project, 2018], bound to the TEE measurement so the record names which agent acted.

## 4.4 Cross-organization flow (Phase 2)

When two runtimes in different trust domains participate (for example a gateway and an upstream server, each in its own TEE), each is an independent Attester. Each produces its own Trust Record, signed by its own `cnf` key and bound to its own platform attestation, and both carry the *same* subject/session identifier. The shared identifier links the two records without sharing private key material; a Verifier confirms the linkage and appraises each record independently, so tampering with one does not affect the other. cMCP [Poddar et al., 2026] demonstrates this as a dual-TEE protocol at the MCP boundary.

# 5 Verification Procedure

Any party (a browser, a CLI, an in-cluster verifier, or a third-party auditor) appraises a Trust Record with no callback to the issuer and no vendor in the trust path beyond the silicon root and the transparency-log operator. The steps are fail-closed and ordered so that nothing is trusted before the signature is checked:

1. **Signature binding.** Verify the record’s signature (Section 3.4) against the key in `cnf`, *before any other field is trusted*. A record with no verifiable binding is rejected.

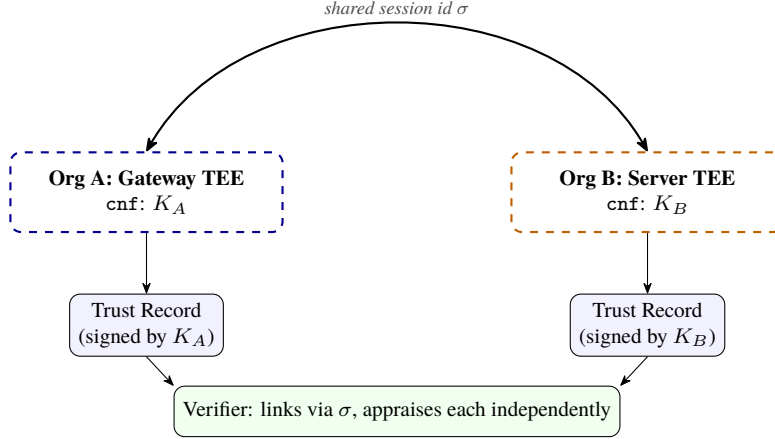


Figure 2: Cross-organization attestation. Two independent Attesters issue separately key-bound Trust Records carrying a shared session identifier; the Verifier links them without either side sharing key material.

2. **Freshness.** Confirm `iat` is within the maximum age (default 24 hours unless the profile overrides). If the verifier issued a challenge nonce, confirm `runtime.nonce` echoes it.
3. **Silicon root.** Confirm the attestation signature chain resolves to a known silicon root (AMD [Advanced Micro Devices, Inc., 2020], Intel [Intel Corporation, 2023], NVIDIA, or equivalent), with unrevoked vendor keys.
4. **Reference values.** Confirm the runtime measurements match the published Reference Integrity Manifests (RIMs).
5. **Policy.** Confirm `policy.bundle_hash` matches the policy the relying party expects, and inspect `enforcement_mode`.
6. **Transparency.** Confirm the transparency receipt resolves on the named append-only log.
7. **Provenance.** Confirm `build_provenance` resolves to a trusted builder at the required SLSA level.

## 5.1 Result object

The verifier emits its own appraisal in the EAR format, separating *what was claimed* from *what was accepted*. The appraisal carries an overall status (affirming, warning, contraindicated, or none) together with a per-claim breakdown of which fields verified and which did not. A field may be *unverified* without the record being *invalid*: for instance, a silicon cert-chain step that needs an online vendor service (AMD KDS for the SEV-SNP VCEK chain, Intel DCAP for the TDX quote) may be unavailable to an offline verifier, which then reports that field as unverified and downgrades the status rather than rejecting outright. A failed *signature* binding or a *stale* record, by contrast, is fatal: the record is rejected. The cMCP reference verifier (`cmcp_verify`) implements exactly this object, with status values `verified`, `partially_verified`, and `unverified` [Poddar et al., 2026].

A Relying Party keys its decision on the appraisal status plus its own policy on which fields are mandatory for its risk tolerance.

## 6 Profiles

TRACE is a base format; a *profile* fixes the platform, the policy engine, and the evidence specifics for a deployment context.

### 6.1 The cMCP profile

cMCP [Poddar et al., 2026] is the reference profile. It produces a Trust Record at the Model Context Protocol [Model Context Protocol Project, 2024] boundary: a TEE gateway evaluates Cedar [Cutler et al., 2024] policies before each tool call, measures the policy bundle and tool catalog at startup, and emits a Trust Record at session close that binds the

session’s hash-chained audit log to the gateway’s attested key. cMCP is an *enveloping*-signature profile (Section 3.4): its RuntimeClaim wraps the canonical record with an Ed25519 signature, and it commits the key into the hardware report\_data via an RFC 7638 JWK thumbprint [Jones and Sakimura, 2015].

Table 2 maps the base Trust Record claims onto the cMCP RuntimeClaim. cMCP populates the runtime-governance core and adds gateway-specific addenda (the hash-chained audit log, per-session call summary, and catalog state); the supply-chain and transparency claims (model, build\_provenance, appraisal, transparency) are not yet populated and are the gap cMCP closes as it matures toward the full Trust Record.

Table 2: Base Trust Record claims as realized by the cMCP RuntimeClaim.

| Trust Record claim                              | cMCP RuntimeClaim location                      | Status                        |
|-------------------------------------------------|-------------------------------------------------|-------------------------------|
| eat_profile, subject runtime                    | iat, trace.* (subject is the gateway SPIFFE ID) | populated                     |
| policy                                          | trace.runtime (platform, measurement, nonce)    | populated                     |
| data_class                                      | trace.policy (Cedar bundle hash, mode)          | populated                     |
| tool_transcript                                 | trace.data_class (session max sensitivity)      | populated                     |
| cnf                                             | trace.tool_transcript (hash = audit-chain tip)  | populated                     |
| signature                                       | trace.cnf.jwk (Ed25519 / OKP)                   | populated                     |
| model                                           | RuntimeClaim envelope signature                 | populated (enveloping)        |
| build_provenance                                | n/a                                             | not yet populated             |
| appraisal                                       | n/a                                             | not yet populated             |
| transparency                                    | (optional NRAS appraisal in runtime context)    | not in the record             |
|                                                 | n/a                                             | TRACE v0.2 registry anchoring |
| <i>cMCP addenda (no base-record equivalent)</i> |                                                 |                               |
| gateway.audit_chain                             | hash-chained audit log root/tip/length          | cMCP extension                |
| gateway.call_summary,                           | per-session counts; pinned catalog              | cMCP extension                |
| gateway.catalog                                 | hash                                            |                               |

cMCP’s experimental results (six security properties validated by reproducible experiments) are reported in [Poddar et al., 2026].

## 6.2 Extensibility

A new profile fixes four things for its deployment context: the *platform* and how its measurement and report\_data map into runtime; the *policy model* and how its bundle hash and enforcement mode populate policy (cMCP uses Cedar [Cutler et al., 2024]; OPA/Rego or a custom DSL are equally admissible, since TRACE binds a policy hash, not a language); the *identity source* for subject (a SPIFFE SVID [SPIFFE Project, 2018] or a DID); and the *signature binding* it declares (embedded or enveloping, Section 3.4). The Agent Governance Toolkit [Microsoft Corporation, 2026] supplies a policy engine and identity layer that a profile can wire into the policy and subject claims directly. An AI-agent profile registry pins, per workload class, the required claim set, evidence requirements, and verification rules.

## 7 Conformance and Reproducibility

TRACE ships an executable conformance suite (trace-tests) and a public transparency registry (trace-registry). Both are open source, so any implementer can check a Trust Record against the normative requirements and any auditor can confirm a record was anchored.

## 7.1 Conformance suite

The suite is organized into seven modules, each validating one claim group, and three conformance levels that compose them (Table 3). It is pip-installable and run from the command line:

```
1 pip install agentrust-trace-tests
2 trace-tests verify --record trust-record.json --level 1 [--max-age SECONDS]
```

Each check reports pass, fail, skip (a benign omission of an optional field), or unverified (the signature could not be cryptographically verified). A record that is unverified passes at Level 0 with an advisory but fails at Level 1 and above; a failed signature or a stale iat fails at every level.

Table 3: Conformance modules and the levels that require them. Level 0 targets software-only development, Level 1 production TEE-attested records, and Level 2 full records with transparency anchoring.

| Module | Validates                                                               | L0 | L1 | L2 |
|--------|-------------------------------------------------------------------------|----|----|----|
| TR-ENV | EAT structure, profile URI, iat freshness, subject, cnf                 | ✓  | ✓  | ✓  |
| TR-SIG | EdDSA signature (EC keys accepted, not verified); no private key in cnf | ✓  | ✓  | ✓  |
| TR-POL | policy bundle-hash format and enforcement mode                          | ✓  | ✓  | ✓  |
| TR-RTE | TEE platform (not software-only), measurement, RIM URI                  |    | ✓  | ✓  |
| TR-SCA | SLSA level and digest format                                            |    | ✓  | ✓  |
| TR-TXN | tool-transcript hash format                                             |    |    | ✓  |
| TR-ANC | transparency claim is an https URI                                      |    |    | ✓  |

The suite is reproducible against a set of versioned test vectors: valid Level 0 and Level 2 records, a cMCP-wrapped record, and negative cases (missing runtime, wrong profile). A conformance claim is therefore checkable, not asserted. The default freshness bound is 24 hours, configurable per deployment. Table 3 describes trace-tests 0.2.0, the evaluated revision: TR-TXN checks the digest format, not the transcript, and TR-ANC does not replay an inclusion proof. Later releases replay the proof against the committed root.

## 7.2 Transparency registry

trace-registry is the public accountability layer. Each entry records the Merkle root [Laurie et al., 2013] of a batch of signed Trust Records as an append-only, git-committed file; the commit history is the tamper-evidence layer, since rewriting a published entry diverges hashes that auditors and mirrors have already observed. A producer issues each record holder an inclusion proof (a leaf index and audit path); a verifier recomputes the leaf hash and replays the inclusion-proof check, accepting only if the recomputed root equals the committed Merkle root. A standalone verify\_inclusion tool performs this offline, so the transparency claim (Section 3) is independently checkable.

## 8 Evaluation

We evaluate the deterministic core of the reference implementation: the parts of a Trust Record whose cost and correctness do not depend on a live model or an online attestation service. All numbers below were produced by a committed harness (experiments/bench.py) run against agentrust-trace at trace-spec revision 523f8cc and trace-tests 0.2.0 (revision 661c177); the raw outputs are committed under experiments/results/. The SDK reported version 0.2.0, but its signing code is that of release 0.3.0: release 0.2.0 canonicalizes with json.dumps(sort\_keys=True) and has no verify\_record, so the harness cannot run against it. The five vectors and the checker at these revisions use the v0.1 profile URI tag:agentrust.io,2026:trace-v0.1; that checker rejects a record carrying the v0.2 URI of Section 3 with TR-ENV-001. Silicon cert-chain appraisal (AMD KDS, Intel DCAP) and any LLM-in-the-loop cost are out of scope here because they require external services or model calls, and are not measured. Measurements were taken on a single core of an ARMv8 (Qualcomm) host under CPython 3.12; absolute latencies are hardware-dependent and are reported to show order of magnitude, not a hardware ranking.

## 8.1 Canonicalization determinism

A Trust Record’s signature is taken over its RFC 8785 JCS canonical byte sequence (Section 3.4). Cross-implementation verification only works if canonicalization is deterministic: the same logical record must yield byte-identical output regardless of how the object was constructed. We tested this directly by canonicalizing each of the five shipped conformance vectors 200 times, shuffling every object’s key insertion order on each pass, and hashing the result. A correct canonicalizer must produce exactly one distinct SHA-256 per record. Across all  $5 \times 200 = 1000$  canonicalizations we observed exactly 5 distinct hashes, one per record (Table 4): canonicalization is insensitive to key order and fully deterministic, as required. Shuffling key order does not distinguish RFC 8785 from sorted `json.dumps`, and the evaluated checker used the latter: a record with a non-ASCII subject that the SDK signs and verifies fails TR-SIG at the same revisions (experiments/results/rerun-2026-10-01). Later trace-tests releases canonicalize with RFC 8785.

## 8.2 Sign and verify cost

Table 4 reports the end-to-end cost of the signing and verification paths, each of which canonicalizes the record (JCS) and then runs Ed25519. Over 5000 iterations after a 500-iteration warmup, signing averaged 0.129 ms per record (median 0.094 ms, p99 0.580 ms) and verification averaged 0.211 ms (median 0.158 ms, p99 0.856 ms), for single-core throughputs of roughly 7,700 signs/s and 4,700 verifies/s. Figure 3 shows the full per-operation latency distributions and Figure 4 the throughput. Sub-millisecond verification means a relying party can appraise the deterministic core of a record inline on the request path; the cost that matters in production is the external attestation and transparency lookups, not the record cryptography.

Table 4: Deterministic-core measurements from experiments/bench.py (agentrust-trace at 523f8cc, trace-tests 0.2.0; single core, CPython 3.12, ARMv8). Latencies are per operation over 5000 iterations.

| Operation                                                                 | mean (ms) | median (ms) | p99 (ms) | throughput (ops/s) |
|---------------------------------------------------------------------------|-----------|-------------|----------|--------------------|
| JCS canonicalize + sign                                                   | 0.129     | 0.094       | 0.580    | 7,734              |
| JCS canonicalize + verify                                                 | 0.211     | 0.158       | 0.856    | 4,731              |
| JCS determinism: 1000 canonicalizations (5 records $\times$ 200 shuffles) |           |             |          |                    |
| distinct hashes observed: 5 (one per record) $\Rightarrow$ deterministic  |           |             |          |                    |

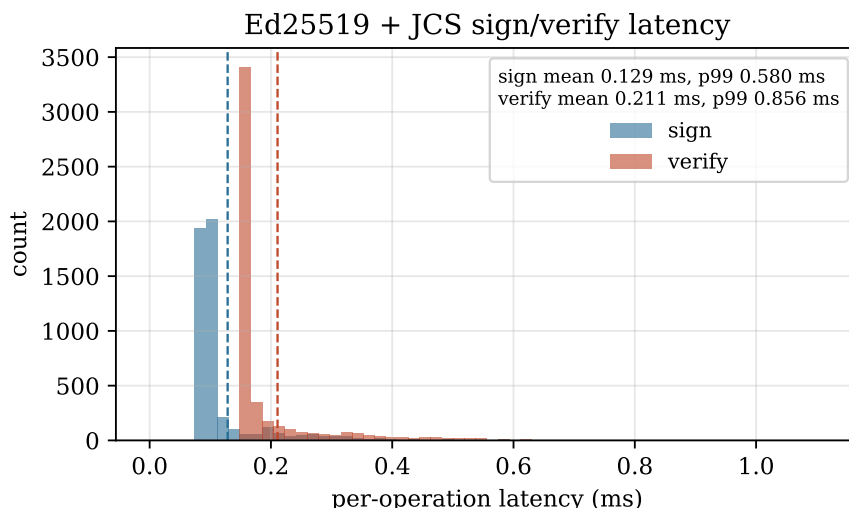


Figure 3: Per-operation latency distribution for the sign and verify paths (JCS canonicalization plus Ed25519), 5000 iterations each. Dashed lines mark the means. Both paths are sub-millisecond in the body of the distribution, with a short scheduler-induced tail.

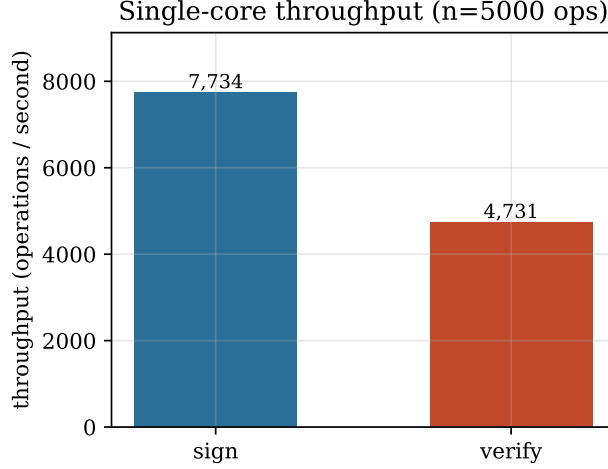


Figure 4: Single-core throughput for the sign and verify paths. Verification is the hot path for a relying party; at roughly 4,700 verifies/s per core the record cryptography is not the bottleneck relative to the external attestation and transparency lookups a full appraisal also performs.

### 8.3 Record size

Trust Records are compact. Table 5 gives the canonical (signature pre-image) size of each shipped vector plus a synthetic, fully populated Level 2 record. A minimal Level 0 record is under 0.9 KB; a complete Level 2 record with TEE measurement, SLSA provenance, tool-transcript binding, and transparency receipt is under 1.4 KB; the cMCP-profiled envelope, which carries the full gateway call summary, is 1.6 KB. The synthetic record was measured with the v0.1 profile URI; the committed harness now emits the v0.2 URI, which is four bytes longer, and gives 1,389 bytes. Records this small travel with the workload as a header or sidecar without meaningful overhead.

Table 5: Canonical (JCS, signature pre-image) size of the shipped conformance vectors and a synthetic fully-populated Level 2 record, from `experiments/results/record_sizes.json`.

| Record                                         | canonical size (bytes) |
|------------------------------------------------|------------------------|
| <code>valid_level0</code> (minimal L0)         | 878                    |
| <code>valid_level0_with_transcript</code>      | 1,109                  |
| synthetic full L2 (all fields)                 | 1,385                  |
| <code>valid_cmcp_runtime</code> (cMCP profile) | 1,612                  |

### 8.4 Conformance suite behaviour

Finally we ran the shipped `trace-tests` runner over every shipped vector at Levels 0, 1, and 2, tallying `pass/fail/skip/unverified` findings (Table 6). The archival vectors are unsigned test data with a 2025 iat, so the run disables the freshness bound to isolate *structural* conformance; TR-SIG correspondingly reports `unverified` for the unsigned plain-trace vectors, which is the honest expected outcome (an `unverified` record passes at Level 0 with an advisory and fails at Level 1 and above, per Section 7). The two plain-trace positive vectors pass at Level 0 and fail at Levels 1 and 2. The cMCP fixture fails at all three levels in this captured run. The `invalid_wrong_profile` fixture fails at every level, while `invalid_missing_runtime` passes at Level 0 and fails at Levels 1 and 2. These are the outcomes in the committed result JSON; fixture names do not establish the expected or observed verdict. In particular, the Level 0 pass for the missing-runtime fixture is a limit of this historical check set, not evidence of complete structural validation. The signed-record path is exercised separately by the sign/verify benchmark. These results describe the recorded package versions and do not certify the current implementation.

Table 6: Conformance findings from the shipped `trace-tests` runner (`experiments/results/conformance.json`), reported as pass/fail/skip/unverified. “L0 pass” is the suite’s overall verdict at Level 0. Freshness disabled to isolate structural conformance of the archival vectors; unsigned plain-trace vectors report TR-SIG `unverified` by design.

| Vector                                 | fmt   | L0 (p/f/s/u) | L1 (p/f/s/u) | L2 (p/f/s/u) |
|----------------------------------------|-------|--------------|--------------|--------------|
| <code>valid_level0</code>              | trace | 7/0/0/1      | 11/1/1/0     | 12/2/1/0     |
| <code>valid_level0_w/transcript</code> | trace | 7/0/0/1      | 11/1/1/0     | 14/1/1/0     |
| <code>valid_cmcp_runtime</code>        | cMCP  | 6/1/0/0      | 8/2/1/0      | 10/3/1/0     |
| <code>invalid_missing_runtime</code>   | trace | 7/0/0/1      | 9/2/0/0      | 10/3/0/0     |
| <code>invalid_wrong_profile</code>     | trace | 6/1/0/1      | 10/2/1/0     | 11/3/1/0     |

## 9 Related Work

**Attestation standards.** TRACE composes rather than replaces. It instantiates RATS [Birkholz et al., 2023] and profiles EAT [Lundblade et al., 2025] for the wire envelope; where platform SDKs (AMD SEV-SNP [Advanced Micro Devices, Inc., 2020], Intel TDX [Intel Corporation, 2023], and GPU/cloud attestation services) each emit their own evidence shape, TRACE maps that evidence into one envelope and adds the governance, identity, and audit claims an agent runtime needs. Unlike a bare platform quote, a Trust Record answers *what ran, under what policy, on what data, and invoking which tools*, not merely *that the environment is genuine*.

**Supply-chain and identity.** TRACE references SLSA provenance for build-time integrity, SPIFFE [SPIFFE Project, 2018] for workload identity bound to the TEE measurement, and AIBOM formats (SPDX/CycloneDX) for component inventory, each by digest from the corresponding claim. The companion Agent Manifest specification [Siddique, 2026] extends identity to the full set of agent-defining artifacts.

**Tamper-evident logs.** The audit commitment and the transparency receipt build on certificate transparency [Laurie et al., 2013], secure audit logging [Schneier and Kelsey, 1999], and the SCITT append-only-log model. TRACE differs in scoping the per-session chain, binding its tip into a hardware-attested record, and registering the record on a transparency log, so a registered record cannot later be altered or replaced unnoticed. Registration alone does not reveal omitted events; that requires an independently expected event set or terminal boundary (Section 10).

**Agent governance.** Output-filtering and guardrail frameworks constrain agent inputs and outputs but do not attest *that* enforcement occurred or under which policy. AGT [Microsoft Corporation, 2026] supplies the policy engine and identity layer; TRACE is the hardware-attested evidence those controls produce, verifiable by a third party after the fact.

## 10 Conclusion

TRACE gives AI agent runtimes a portable, hardware-attestable evidence record, the Trust Record, that a relying party can appraise offline, without trusting the operator. By instantiating the RATS roles and profiling EAT, it reuses established attestation machinery while adding the model, runtime, policy, data-class, tool-transcript, provenance, appraisal, and transparency claims an agent deployment must account for. A confirmation key identifies the signing key; verified attestation must bind that key to a hardware root before silicon provenance can be claimed. The signed body makes modification detectable. An inclusion proof establishes membership in a particular tree, not completeness of execution or absence of omitted events. Completeness requires an independently expected event set or terminal boundary. The historical fixture outcomes in Section 8 also show limits of the software conformance checks.

**Limitations and future work.** The base format is settled, but several pieces are deliberately staged. Silicon cert-chain appraisal (AMD KDS, Intel DCAP) is an online step a fully offline verifier reports as unverified rather than fatal. The external-execution-evidence receipt format (Section 3) is specified but not yet exercised by the conformance suite. Profile coverage is growing: cMCP is the reference profile at the MCP boundary, with A2A and additional platform

profiles planned. As more profiles and vendor attestation surfaces are mapped in, the registry and conformance levels give the ecosystem a shared, testable definition of what a trustworthy agent execution record is.

## References

- Advanced Micro Devices, Inc. AMD SEV-SNP: Strengthening VM isolation with integrity protection and more. Technical report, AMD, 2020. URL <https://www.amd.com/content/dam/amd/en/documents/epyc-technical-docs/white-papers/SEV-SNP-strengthening-vm-isolation-with-integrity-protection-and-more.pdf>.
- Henk Birkholz, Dave Thaler, Michael Richardson, Ned Smith, and Wei Pan. Remote attestation procedures (RATS) architecture. RFC 9334, 2023. URL <https://www.rfc-editor.org/rfc/rfc9334>.
- Joseph W. Cutler, Craig Disselkoen, Aaron Eline, Shaobo He, Kyle Headley, Michael Hicks, Kesha Hietala, Eleftherios Ioannidis, John Kastner, Anwar Mamat, Darin McAdams, Matt McCutchen, Neha Rungta, Emina Torlak, and Andrew M. Wells. Cedar: A new language for expressive, fast, safe, and analyzable authorization. *Proc. ACM Program. Lang. (OOPSLA)*, 8, 2024. doi: 10.1145/3649835. URL <https://dl.acm.org/doi/10.1145/3649835>.
- Intel Corporation. Intel trust domain extensions (Intel TDX) module architecture specification. Technical report, Intel, 2023. URL <https://www.intel.com/content/www/us/en/developer/tools/trust-domain-extensions/documentation.html>.
- Michael B. Jones and Nat Sakimura. JSON web key (JWK) thumbprint. RFC 7638, 2015. URL <https://www.rfc-editor.org/rfc/rfc7638>.
- Ben Laurie, Adam Langley, and Emilia Kasper. Certificate transparency. RFC 6962, 2013. URL <https://www.rfc-editor.org/rfc/rfc6962>.
- Laurence Lundblade, Giridhar Mandyam, Jeremy O’Donoghue, and Carl Wallace. The entity attestation token (EAT). RFC 9711, 2025. URL <https://www.rfc-editor.org/rfc/rfc9711>.
- Microsoft Corporation. Agent governance toolkit (AGT), 2026. URL <https://github.com/microsoft/agent-governance-toolkit>. Accessed June 2026.
- Model Context Protocol Project. Model context protocol specification, 2024. URL <https://modelcontextprotocol.io/specification>. Accessed June 2026.
- Rishabh Poddar, Aaron Fulkerson, and Imran Siddique. cMCP: Verifiable policy enforcement for AI agent tool calls. Technical report, version 1, OPAQUE Systems, 2026. URL <https://doi.org/10.5281/zenodo.23001694>.
- Bruce Schneier and John Kelsey. Secure audit logs to support computer forensics. *ACM Transactions on Information and System Security*, 2(2), 1999. URL <https://dl.acm.org/doi/10.1145/317087.317089>.
- Imran Siddique. Agent manifest: Portable, hardware-attestable identity and provenance for AI agents. Technical report, version 1, OPAQUE Systems, 2026. URL <https://doi.org/10.5281/zenodo.23001698>.
- SPIFFE Project. SPIFFE: Secure production identity framework for everyone, 2018. URL <https://spiffe.io/docs/latest/spiffe-about/overview/>. Accessed June 2026.